

Lecture 5

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- loss of significance
- cancellation
- recall linear algebra
- cost analysis of basic operations
- identify basic solution schemes to systems

Floating Point Arithmetic

Roundoff errors and floating-point arithmetic

One of the most serious problems in floating point arithmetic is that of cancellation. If two large and close-by numbers are subtracted the result (a small number) carries very few accurate digits (why?). This is fine if the result is not reused. If the result is part of another calculation, then there may be a serious problem

Example

Roots of the equation

$$x^2 + 2px - q = 0$$

Assume we want the root with smallest absolute value:

$$y = -p + \sqrt{p^2 + q} = \frac{q}{p + \sqrt{p^2 + q}}$$

Loss of Significance

Listing 1: alg 1

```
1  $s := p^2$   
2  $t := s + q$   
3  $u := \sqrt{t}$   
4  $y := -p + u$ 
```

Listing 2: alg 2

```
1  $s := p^2$   
2  $t := s + q$   
3  $u := \sqrt{t}$   
4  $v := p + u$   
5  $y := q/v$ 
```

Step 4 of Algorithm 1 can have severe cancellation when $p \gg q$ and when p is positive.

```

1 >> p = 10000;
2 >> q = 1;
3 >> y = -p+sqrt(p^2 + q)
4 y = 5.0000000055588316e-05
5
6 >> y2 = q/(p+sqrt(p^2+q))
7 y2 = 4.9999999875000000e-05
8
9 >> x = y;
10 >> x^2 + 2 * p * x - q
11 ans = 1.361766321927860e-08
12
13 >> x = y2;
14 >> x^2 + 2 * p * x - q
15 ans = -1.110223024625157e-16

```

Consider now the case when

$$p = -(1 + \epsilon/2) \quad \text{and} \quad q = -(1 + \epsilon)$$

Exact Roots? Take $\epsilon = 1.E - 08$ and use matlab:

```

1 x = - p - sqrt(p^2 + q)    --->    1.000000000500000
2 y = - p + sqrt(p^2 + q)    --->    1.000000000500000

```

Catastrophic Cancellation

Adding $c = a + b$ will result in a large error if

- $a \gg b$
- $a \ll b$

Let

$$a = x.xxx \cdots \times 10^0$$

$$b = y.yyy \cdots \times 10^{-8}$$

Then

$$\begin{array}{r} \text{finite precision} \\ \overbrace{x.xxx \ xxxx \ xxxx \ xxxx} \\ + \quad 0.000 \ 0000 \ yyy \ yyy \ yyy \ yyy \\ \hline = \quad x.xxx \ xxxx \ zzzz \ zzzz \quad \underbrace{yyyy \ yyy \ yyy \ yyy}_{\text{lost precision}} \end{array}$$

Catastrophic Cancellation

Subtracting $c = a - b$ will result in large error if $a \approx b$. For example

$$\begin{array}{r}
 ax.xxxx\ xxxx\ xxx1 \quad \overbrace{ssss \dots}^{\text{lost}} \\
 bx.xxxx\ xxxx\ xxx0 \quad \overbrace{tttt \dots}^{\text{lost}}
 \end{array}$$

Then

$$\begin{array}{r}
 \quad \quad \quad \overbrace{x.xxx\ xxxx\ xxx1}^{\text{finite precision}} \\
 + \quad \overbrace{x.xxx\ xxxx\ xxx0}^{\text{finite precision}} \\
 \hline
 = \quad 0.000\ 0000\ 0001 \quad \underbrace{????\ ????}_{\text{lost precision}}
 \end{array}$$

Summary

- addition: $c = a + b$ if $a \gg b$ or $a \ll b$
- subtraction: $c = a - b$ if $a \approx b$
- catastrophic: caused by a single operation, not by an accumulation of errors
- can often be fixed by mathematical rearrangement

Loss of Significance

Example

$x = 0.3721448693$ and $y = 0.3720214371$. What is the relative error in $x - y$ in a computer with 5 decimal digits of accuracy?

$$\frac{|x - y - (\bar{x} - \bar{y})|}{|x - y|} = \frac{|0.3721448693 - 0.3720214371 - 0.37214 + 0.37202|}{|0.3721448693 - 0.3720214371|}$$
$$\approx 3 \times 10^{-2}$$

Loss of Significance

Loss of Precision Theorem

Let x and y be (normalized) floating point machine numbers with $x > y > 0$.

If $2^{-p} \leq 1 - \frac{y}{x} \leq 2^{-q}$ for positive integers p and q , the significant binary digits lost in calculating $x - y$ is between q and p .

Loss of Significance

Example

Consider $x = 37.593621$ and $y = 37.584216$.

$$2^{-11} < 1 - \frac{y}{x} = 0.0002501754 < 2^{-12}$$

So we lose 11 or 12 bits in the computation of $x - y$. yikes!

Example

Back to the other example (5 digits): $x = 0.37214$ and $y = 0.37202$.

$$10^{-4} < 1 - \frac{y}{x} = 0.00032 < 10^{-5}$$

So we lose 4 or 5 bits in the computation of $x - y$. Here, $x - y = 0.00012$ which has only 1 significant digit that we can be sure about

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Problem at $x \approx 0$.

Loss of Significance

So what to do? Mainly rearrangement.

$$f(x) = \sqrt{x^2 + 1} - 1$$

Problem at $x \approx 0$.

One type of fix:

$$\begin{aligned} f(x) &= \left(\sqrt{x^2 + 1} - 1 \right) \left(\frac{\sqrt{x^2 + 1} + 1}{\sqrt{x^2 + 1} + 1} \right) \\ &= \frac{x^2}{\sqrt{x^2 + 1} + 1} \end{aligned}$$

no subtraction!

Why this is important: matrix computation

- matrix problems arise in many areas of engineering, computer science, information sciences, graphics, design, etc
- Basic Linear Algebra Subprograms (BLAS) is an interface standard for operations
- simple systems set the stage for further development: avoiding error, avoiding large costs

Prereq

Linear Algebra is a prerequisite of the course!

Matrix Inverse

Let A be a square (i.e. $n \times n$) with real elements. The *inverse* of A is designated A^{-1} , and has the property that

$$A^{-1}A = I \quad \text{and} \quad AA^{-1} = I$$

The **formal solution** to $Ax = b$ is $x = A^{-1}b$.

$$Ax = b$$

$$A^{-1}Ax = A^{-1}b$$

$$Ix = A^{-1}b$$

$$x = A^{-1}b$$

Matrix Inverse

- formal solution to $Ax = b$ is $x = A^{-1}b$

Matrix Inverse

- formal solution to $Ax = b$ is $x = A^{-1}b$
- BUT it is *bad* evaluate x this way

Matrix Inverse

- formal solution to $Ax = b$ is $x = A^{-1}b$
- BUT it is *bad* evaluate x this way
- why?

Matrix Inverse

- formal solution to $Ax = b$ is $x = A^{-1}b$
- BUT it is *bad* evaluate x this way
- why?
- we will not form A^{-1} , but solve for x directly using Gaussian elimination.

Why do we care as Numerical Analysts?

Open questions:

- How *expensive* is it to solve $Ax = b$?
- What problems (errors) will we encounter solving $Ax = b$?
- Some matrices are easy/cheap to use: diagonal, tridiagonal, etc.
 - are there others? what makes something a "good" matrix numerically?
 - are there bad ones? how do we identify them numerically?
- what do actual numerical analysts, engineers, developers, etc use?!?!?

Formal Solution when A is $n \times n$

The *formal solution* to $Ax = b$ is

$$x = A^{-1}b$$

where A is $n \times n$.

If A^{-1} exists then A is said to be **nonsingular**.

If A^{-1} does not exist then A is said to be **singular**.

Formal Solution when A is $n \times n$

If A^{-1} exists then

$$Ax = b \quad \implies \quad x = A^{-1}b$$

but

Do not compute the solution to $Ax = b$ by finding A^{-1} , and then multiplying b by A^{-1} !

We see: $x = A^{-1}b$

We do: Solve $Ax = b$ by Gaussian elimination or an equivalent algorithm

Singularity of A

If an $n \times n$ matrix, A , is **singular** then

- the columns of A are linearly dependent
- the rows of A are linearly dependent
- $\text{rank}(A) < n$
- $\det(A) = 0$
- A^{-1} does not exist
- a solution to $Ax = b$ may not exist
- If a solution to $Ax = b$ exists, it is not unique

Summary of Requirements for Solution of $Ax = b$

Given the $n \times n$ matrix A and the $n \times 1$ vector, b

- the solution to $Ax = b$ exists and is unique for any b if and only if $\text{rank}(A) = n$.

Recall: $\text{rank} = \#$ of linearly independent rows or columns

Recall: $\text{Range}(A) = \text{set of vectors } y \text{ such that } Ax = y \text{ for some } x$

Solving a system

$$Ax = b$$

Three situations:

- ① A is nonsingular: There exists a unique solution $x = A^{-1}b$
- ② A is singular and $b \in \text{Range}(A)$: There are infinite solutions.
- ③ A is singular and $b \notin \text{Range}(A)$: There no solutions.

① $A = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$ $b = \begin{bmatrix} 1 \\ 8 \end{bmatrix}$, then $x = \begin{bmatrix} 1/2 \\ 2 \end{bmatrix}$.

② $A = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$ $b = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, then infinitely many solutions. $x = \begin{bmatrix} 1/2 \\ \alpha \end{bmatrix}$.

③ $A = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$ $b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, then no solutions.

What's the big deal?cost

Look at 1D:

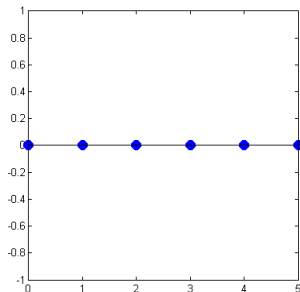
- 3 equations
- 3 unknowns
- each unknown coupled to its neighbor

$$\begin{array}{rrcr} 2x_1 & -x_2 & & = 5.8 \\ -x_1 & 2x_2 & -x_3 & = 13.9 \\ & -x_2 & 2x_3 & = 0.03 \end{array}$$

or

$$\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 5.8 \\ 13.9 \\ 0.03 \end{bmatrix}$$

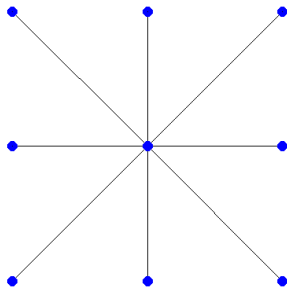
Easy in 1d



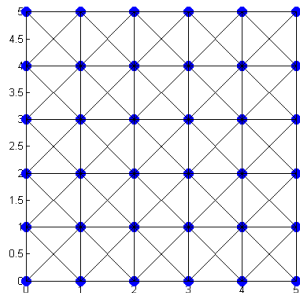
$$\begin{bmatrix} 2 & -1 & & & & \\ & \ddots & & & & \\ & & -1 & 2 & -1 & \\ & & & \ddots & & \\ & & & & -1 & 2 \end{bmatrix}$$

- n points in the grid
- $3 * (n - 2) + 2 + 2$ or about $3n$ nonzeros in the matrix
- trididagonal (easy)

2D: harder

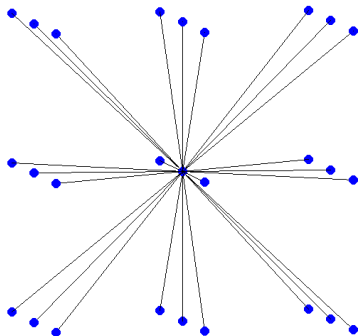


$$\begin{bmatrix} 8 & -1 & & -1 & & \\ & \ddots & & & & \\ -1 & -1 & 8 & -1 & -1 & \\ & & & \ddots & & \\ & -1 & & & -1 & 8 \end{bmatrix}$$

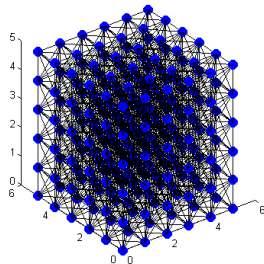


- n points in one direction
- n^2 points in grid
- about 9 nonzeros in each row
- about $9n^2$ nonzeros in the matrix
- n -banded (harder...we will see)

3D: hardest

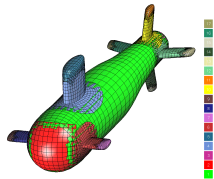


- n points in one direction
- n^3 points in grid

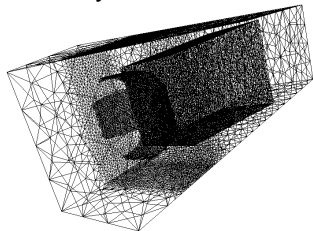


- about 27 nonzeros in each row
- about $27n^3$ nonzeros in the matrix
- n^2 -banded (yikes!)

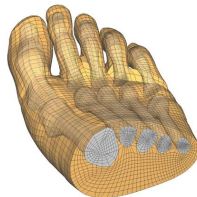
Applications get harder and harder...



courtesy of LLNL

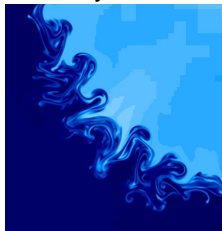


courtesy of Rice



Posterior view of the forefoot mesh
and bones

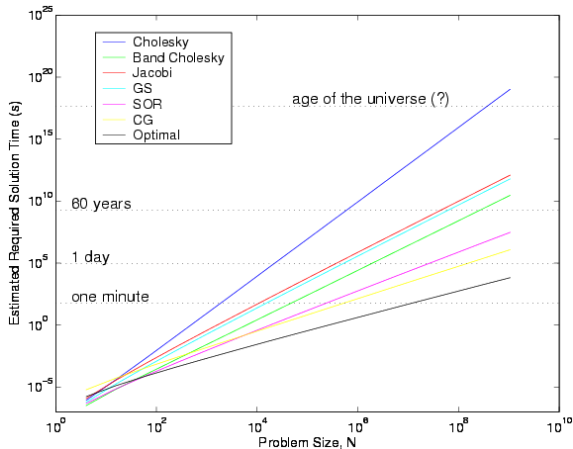
courtesy of TrueGrid



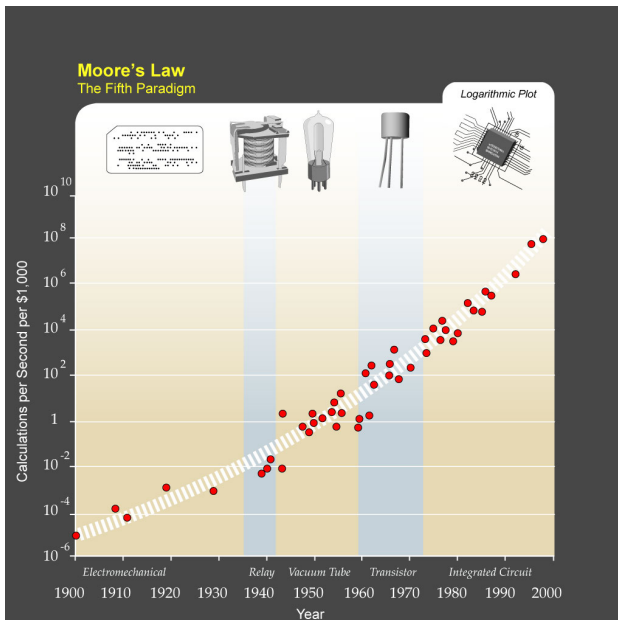
courtesy of Warwick U.

Solving is a problem...

dim	unknowns	storage	example (n)		
1D	n	$3n$	10	100	1000
2D	n^2	$9n^2$	10^2	10^4	10^6
3D	n^3	$27n^3$	10^3	10^6	10^9



Moore...



What's the problem?

- humans: milliFLOPS
- hand calculators: 10 FLOPS
- desktops: a few GFLOPS (10^9 FLOPS)

look at the basic operations!

BLAS

Basic Linear Algebra Subprograms (BLAS) interface introduced APIs for common linear algebra tasks

- Level 1: vector operations (dot products, vector norms, etc) e.g.

$$y \leftarrow \alpha x + y$$

- Level 2: matrix-vector operations, e.g.

$$y \leftarrow \alpha Ax + By$$

- Level 3: matrix-matrix operations, e.g.

$$C \leftarrow \alpha AB + \beta C$$

- optimized versions of the reference BLAS are used everyday: ATLAS, etc.

vec-vec, mat-vec, mat-mat

- inner product of u and v both $[n \times 1]$

$$\sigma = u^T v = u_1 v_1 + \cdots + u_n v_n$$

- $\rightarrow n$ multiplies, $n - 1$ additions
- $\rightarrow \mathcal{O}(n)$ flops

vec-vec, mat-vec, mat-mat

- mat-vec of A ($[n \times n]$) and u ($[n \times 1]$)

```
1   for  $i = 1, \dots, n$ 
2       for  $j = 1, \dots, n$ 
3            $v(i) = a(i, j)u(j) + v(i)$ 
4       end
5   end
```

- $\rightarrow n^2$ multiplies, n^2 additions
- $\rightarrow \mathcal{O}(n^2)$ flops

vec-vec, mat-vec, mat-mat

- mat-mat of A ($[n \times n]$) and B ($[n \times n]$)

```
1   for  $j = 1, \dots, n$ 
2       for  $i = 1, \dots, n$ 
3           for  $k = 1, \dots, n$ 
4                $C(k, j) = A(k, i)B(i, j) + C(k, j)$ 
5           end
6       end
7   end
```

- $\rightarrow n^3$ multiplies, n^3 additions
- $\rightarrow \mathcal{O}(n^3)$ flops

vec-vec, mat-vec, mat-mat

Operation	FLOPS
$u^T v$	$\mathcal{O}(n)$
Au	$\mathcal{O}(n^2)$
AB	$\mathcal{O}(n^3)$

matlab test

four tests:

- matrix-matrix multiply
- inner product
- matrix-vector multiply
- n matrix-vector multiplies

order them...fastest to slowest

testflop.m

Gaussian Elimination

- Solving Diagonal Systems
- Solving Triangular Systems
- Gaussian Elimination Without Pivoting
 - Hand Calculations
 - Cartoon Version
 - The Algorithm
- Gaussian Elimination with Pivoting
 - Row or Column Interchanges, or Both
 - Implementation
- Solving Systems with the Backslash Operator

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

is equivalent to

$$\begin{aligned} x_1 &= -1 \\ 3x_2 &= 6 \\ 5x_3 &= -15 \end{aligned}$$

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

is equivalent to

$$\begin{aligned} x_1 &= -1 \\ 3x_2 &= 6 \\ 5x_3 &= -15 \end{aligned}$$

The solution is

$$x_1 = -1 \qquad x_2 = \frac{6}{3} = 2 \qquad x_3 = \frac{-15}{5} = -3$$

Solving Diagonal Systems

Listing 3: Diagonal System Solution

```
1 given A, b
2 for i = 1...n
3      $x_i = b_i / a_{i,i}$ 
4 end
```

In Matlab:

```
1 >> A = ...           % A is a diagonal matrix
2 >> b = ...
3 >> x = b ./ diag(A)
```

This is the *only* place where element-by-element division (`./`) has anything to do with solving linear systems of equations.

Operations?

Try...

Sketch out an operation count to solve a diagonal system of equations...

Operations?

Try...

Sketch out an operation count to solve a diagonal system of equations...

cheap!

one division n times $\longrightarrow \mathcal{O}(n)$ FLOPS

Triangular Systems

The generic lower and upper triangular matrices are

$$L = \begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & & 0 \\ \vdots & & \ddots & \vdots \\ l_{n1} & & \cdots & l_{nn} \end{bmatrix}$$

and

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & & u_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & & \cdots & u_{nn} \end{bmatrix}$$

The triangular systems

$$Ly = b \quad Ux = c$$

are easily solved by **forward substitution** and **backward substitution**, respectively

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

is equivalent to

$$\begin{array}{rcrcrcrcrcr} -2x_1 & + & x_2 & + & 2x_3 & = & 9 \\ & & 3x_2 & + & -2x_3 & = & -1 \\ & & & & 4x_3 & = & 8 \end{array}$$

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

is equivalent to

$$\begin{array}{rclcl} -2x_1 & + & x_2 & + & 2x_3 & = & 9 \\ & & 3x_2 & + & -2x_3 & = & -1 \\ & & & & 4x_3 & = & 8 \end{array}$$

Solve in backward order (last equation is solved first)

$$x_3 = \frac{8}{4} = 2$$

$$x_2 = \frac{1}{3}(-1 + 2x_3) = \frac{3}{3} = 1$$

$$x_1 = \frac{1}{-2}(9 - x_2 - 2x_3) = \frac{4}{-2} = -2$$

Solving Triangular Systems

Solving for $x_1, x_2, \dots, x_n$ for a lower triangular system is called **forward substitution**.

```
1  given  $L, b$ 
2   $x_1 = b_1 / \ell_{11}$ 
3  for  $i = 2 \dots n$ 
4       $s = b_i$ 
5      for  $j = 1 \dots i - 1$ 
6           $s = s - \ell_{i,j} x_j$ 
7      end
8       $x_i = s / \ell_{i,i}$ 
9  end
```

Solving Triangular Systems

Solving for $x_1, x_2, \dots, x_n$ for a lower triangular system is called **forward substitution**.

```
1  given  $L, b$ 
2   $x_1 = b_1 / \ell_{11}$ 
3  for  $i = 2 \dots n$ 
4       $s = b_i$ 
5      for  $j = 1 \dots i - 1$ 
6           $s = s - \ell_{i,j} x_j$ 
7      end
8       $x_i = s / \ell_{i,i}$ 
9  end
```

Using forward or backward substitution is sometimes referred to as performing a **triangular solve**.

Operations?

Try...

Sketch out an operation count to solve a triangular system of equations...

Operations?

Try...

Sketch out an operation count to solve a triangular system of equations...

cheap!

- begin in the bottom corner: 1 div
- row -2: 1 mult, 1 add, 1 div, or 3 FLOPS
- row -3: 2 mult, 2 add, 1 div, or 5 FLOPS
- row -4: 3 mult, 3 add, 1 div, or 7 FLOPS
- $\vdots$
- row - j : about $2j$ FLOPS

Total FLOPS? $\sum_{j=1}^n 2j = 2\frac{n(n+1)}{2}$ or $\mathcal{O}(n^2)$ FLOPS